

DU CAPTEUR À L'ACTIONNEUR

Système embarqué avec capteur ultrason et servo-moteur

Rapport de Projet - 2025

Kiran BONHOMME

Anaïs DIGOUT

Promotion E4e - SE

Mai 2025

Sommaire

- 1. Introduction
 - 1.1 Objectif du projet
 - 1.2 Modes de fonctionnement
 - 1.3 Machine à état
 - 1.4 Comportements supplémentaires
- 2. Conception
 - 2.1 Diagramme de classe
 - 2.2 Diagrammes de séquence
 - Diagramme de séquence principal
 - Traitement des commandes utilisateur
 - Mise à jour du système selon l'état
 - Mode 1 - Suivi de distance
 - Mode 2 - Consigne utilisateur
 - Gestion des threads sur Raspberry Pi
 - 2.4 Brochage des différents éléments
 - Brochage du capteur HC-SR04
 - Brochage du servo-moteur
 - Brochage des LEDs
 - Brochage de la Communication série
- 3. Modules développés
 - 3.1 Module HC-SR04

- 3.2 Module Servo-moteur
- 3.3 Module UART
- 3.4 Fonctions de démonstration
- 3.5 Application principale
- 3.6 Modules Raspberry Pi
 - 3.6.1 Module GPIO
 - 3.6.2 Module Serial
 - 3.6.3 Module Servo
 - 3.6.4 Module Main
- 4. Difficultés rencontrées et solutions
- 5. Interaction Raspberry Pi - STM32
 - 5.1 Architecture du système global
 - 5.2 Fonctionnalités de la Raspberry Pi
 - 5.3 Modules logiciels Raspberry Pi
 - 5.4 Modes de fonctionnement
 - 5.5 Protocole de communication
 - 5.6 Implémentation multi-thread
 - 5.7 Outils de diagnostic
- 6. Conclusion et perspectives
 - Améliorations futures
- 7. Démonstration

1. Introduction

1.1 Objectif du projet

Ce projet vise à développer un système embarqué sur microcontrôleur STM32F407 VGT permettant d'interfacer un capteur de distance à ultrasons (HC-SR04) avec un servo-moteur. L'objectif principal est de créer un système capable de réagir à l'environnement (mesure de distance) ou à des commandes utilisateur (via liaison série) pour positionner précisément un servo-moteur.

Points clés du projet :

- Système distribué entre STM32F4 et Raspberry Pi, communiquant via UART
- Un servo-moteur connecté à chaque plateforme (STM32F4 et Raspberry Pi)
- Capteur ultrason HC-SR04 connecté à la STM32F4
- Interface utilisateur via console sur la Raspberry Pi
- Plusieurs modes de fonctionnement configurables

Le système met en œuvre différentes technologies et compétences en développement embarqué :

- Mesure de distance par ultrasons
- Génération de signaux PWM pour contrôler des servo-moteurs
- Communication série formatée entre microcontrôleur et ordinateur
- Programmation multi-thread sur Raspberry Pi
- Gestion d'une machine à états
- Développement modulaire avec séparation des responsabilités
- Communication entre STM32 et Raspberry Pi
- Interface utilisateur en mode console

1.2 Modes de fonctionnement

Le système propose trois modes de fonctionnement distincts :

1. Mode IDLE (repos) :

- Les LEDs sont éteintes sur la STM32
- Les deux servo-moteurs (STM32 et Raspberry Pi) sont positionnés au centre (position neutre)
- Le système attend une commande utilisateur

2. Mode 1 (asservissement par capteur) :

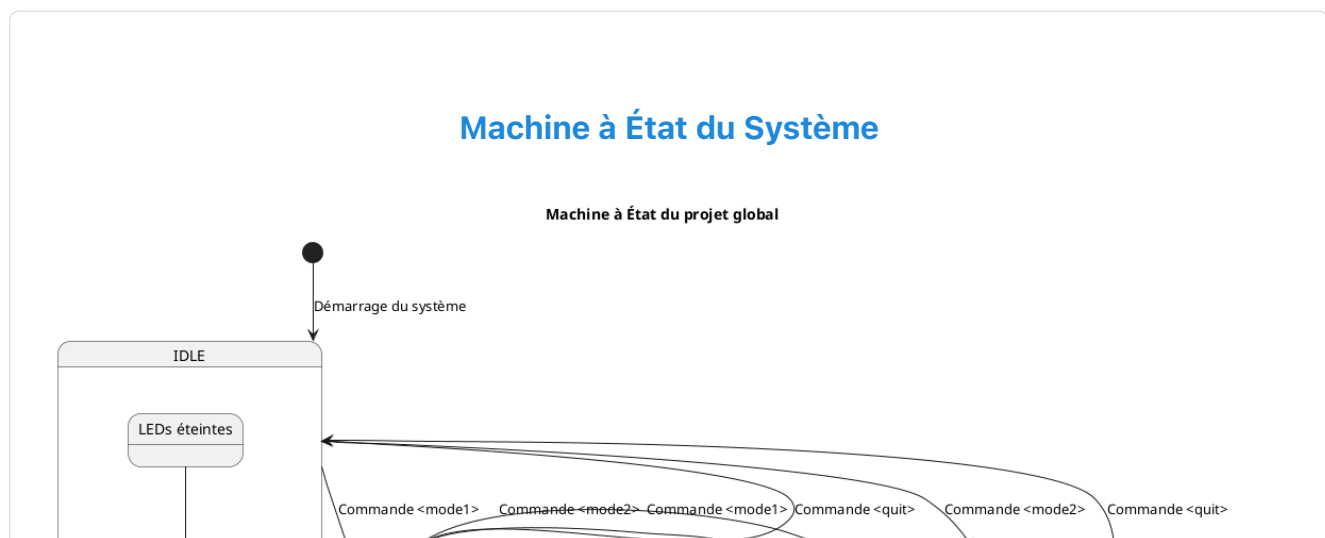
- La LED bleue est allumée sur la STM32 pour indiquer ce mode
- Les deux servo-moteurs (STM32 et Raspberry Pi) se positionnent proportionnellement à la distance mesurée par le capteur HC-SR04
- Plage de mesure valide : 5 à 25 cm
- Si la distance est hors de cette plage, les servo-moteurs reviennent en position centrale
- La STM32 envoie régulièrement les mesures de distance à la Raspberry Pi

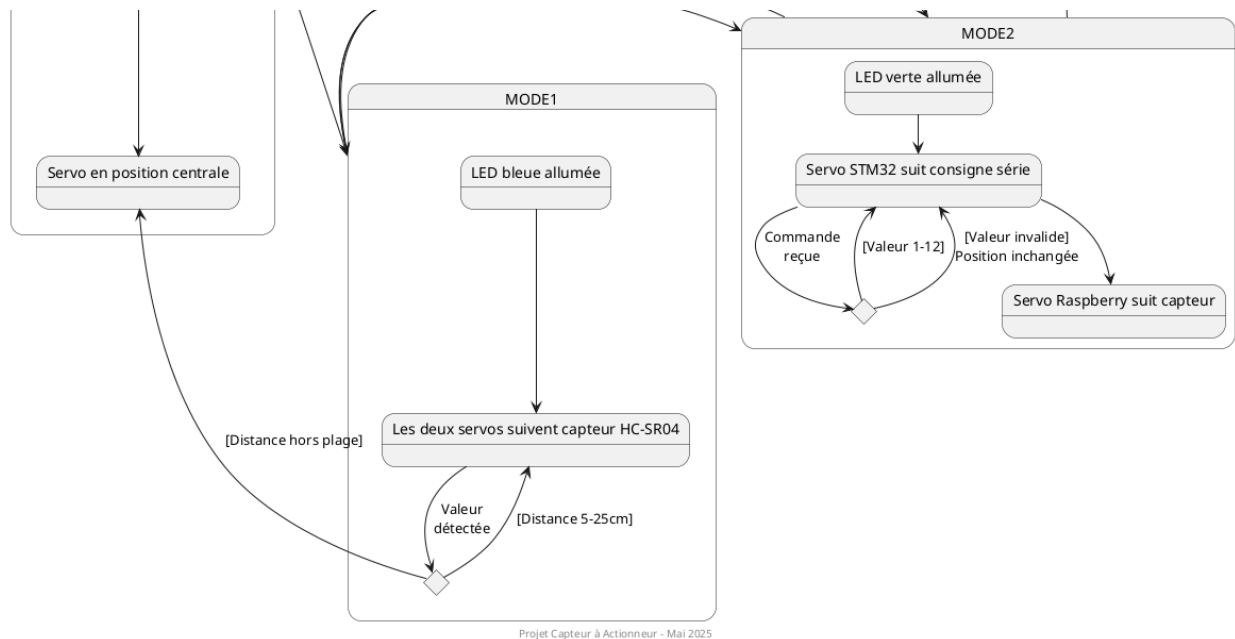
3. Mode 2 (contrôle par commande série) :

- La LED verte est allumée sur la STM32 pour indiquer ce mode
- Le servo-moteur de la STM32 se positionne selon une valeur envoyée par l'utilisateur via la console de la Raspberry Pi
- Le servo-moteur de la Raspberry Pi continue de suivre la distance mesurée par le capteur
- Plage de valeurs acceptées pour le servo STM32 : 1 à 12 (correspondant à différentes positions angulaires)

1.3 Machine à état

Le système est géré par une machine à états dont les transitions sont déclenchées par les commandes utilisateur. Voici le diagramme d'états correspondant :





Ce diagramme montre les trois états principaux (IDLE, MODE1, MODE2), leurs transitions et les comportements associés à chaque état.

Ce diagramme illustre :

- Les trois états principaux du système (IDLE, MODE1, MODE2)
- Les transitions entre états déclenchées par les commandes utilisateur via la Raspberry Pi
- Les comportements spécifiques à chaque état, y compris le comportement des LEDs et des servo-moteurs
- La gestion des cas particuliers comme les mesures hors plage
- Les règles de positionnement des servo-moteurs selon le mode actif

1.4 Comportements supplémentaires

En plus des modes de fonctionnement principaux, le système offre les caractéristiques suivantes :

- **Protocole de communication série structuré :**
 - Les messages sont encapsulés entre délimiteurs '<' et '>' pour garantir l'intégrité des commandes
 - Écho des caractères reçus pour confirmer la réception
 - Messages d'état et d'erreur formatés

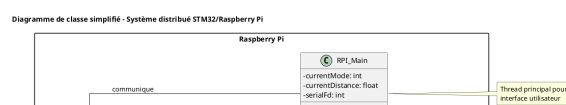
- Transmission régulière des mesures de distance de la STM32 vers la Raspberry Pi
- **Interface utilisateur sur Raspberry Pi :**
 - Interface en ligne de commande pour contrôler le système
 - Affichage des informations de distance et d'état
 - Menu d'aide intégré pour guider l'utilisateur
 - Commandes intuitives pour sélectionner les modes et positionner les servo-moteurs
- **Architecture multi-thread sur Raspberry Pi :**
 - Thread dédié à la génération du signal PWM pour le servo-moteur
 - Thread pour recevoir les données du STM32 en temps réel
 - Thread pour contrôler le servo-moteur selon le mode sélectionné
 - Thread principal pour l'interface utilisateur
- **Gestion des erreurs :**
 - Détection des valeurs hors plage pour le capteur
 - Vérification de la validité des commandes série
 - Messages d'erreur explicites pour guider l'utilisateur
 - Mécanismes de récupération en cas de déconnexion
- **Retour d'information :**
 - Affichage régulier de la distance mesurée
 - Confirmation des commandes reçues
 - Indication visuelle du mode actif par LEDs sur la STM32
 - Messages d'état détaillés sur la console de la Raspberry Pi

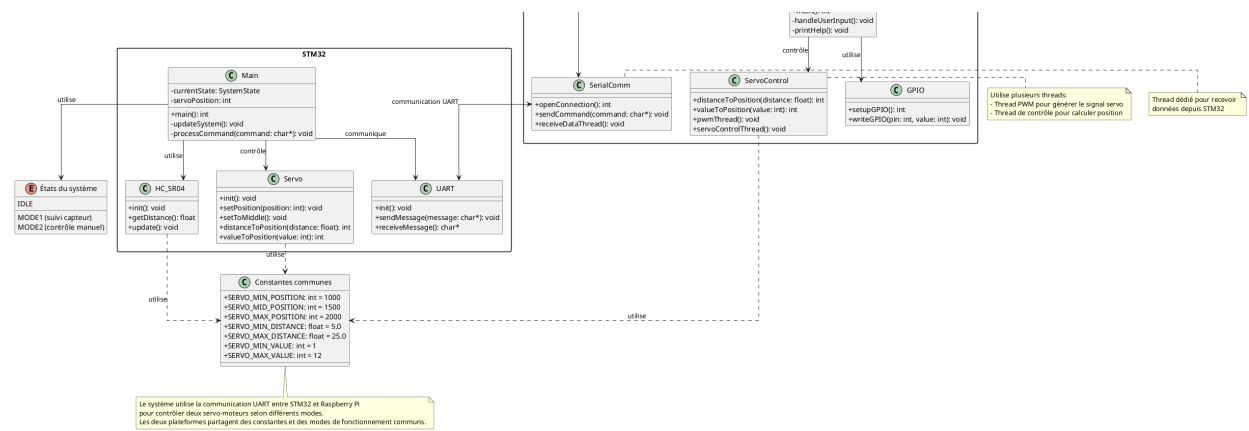
2. Conception

2.1 Diagramme de classe

Le diagramme suivant illustre la structure du code et les relations entre les différents modules du système :

Diagramme de Classe du Système





Ce diagramme présente l'organisation modulaire du système, montrant les classes principales et leurs relations.

Ce diagramme met en évidence :

- La structure modulaire du système avec une séparation claire des responsabilités
- Les différentes classes et leurs méthodes principales
- L'implémentation du pattern Singleton pour le capteur HC-SR04
- Les interfaces entre le contrôleur principal et les différents modules périphériques
- Les constantes et limitations définies pour chaque sous-système
- Les énumérations utilisées pour représenter les états et commandes

Bien que ce diagramme se concentre principalement sur la partie STM32, le système sur Raspberry Pi adopte également une approche modulaire avec des modules distincts pour la communication série, le contrôle du servo-moteur, et la gestion des GPIO.

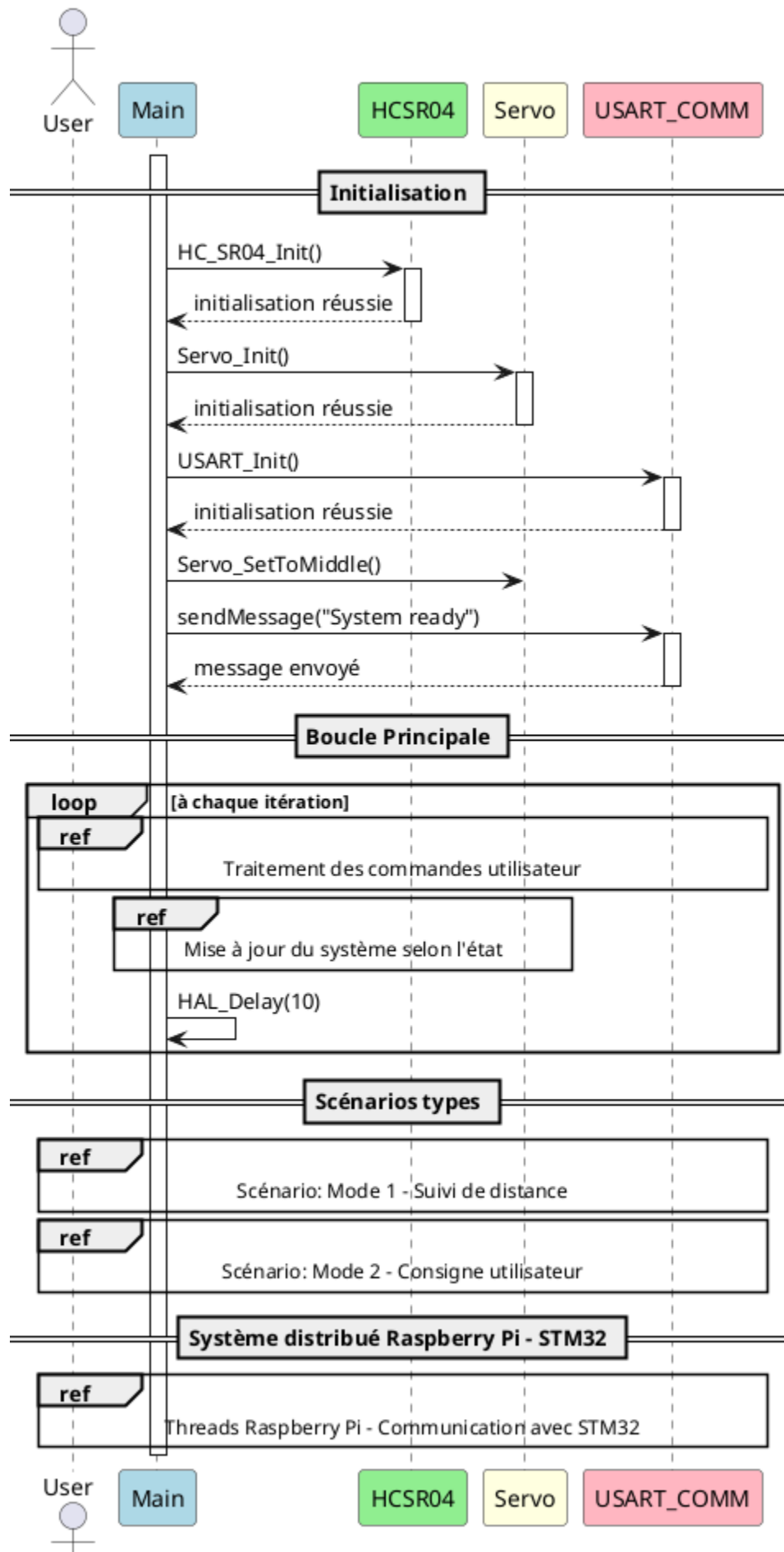
2.2 Diagrammes de séquence

Les diagrammes de séquence suivants montrent les interactions entre les différents modules du système :

Diagramme de séquence principal

Diagramme de Séquence Principal

Diagramme de séquence principal du projet





Vue d'ensemble des interactions dans le système avec références vers les diagrammes plus détaillés.

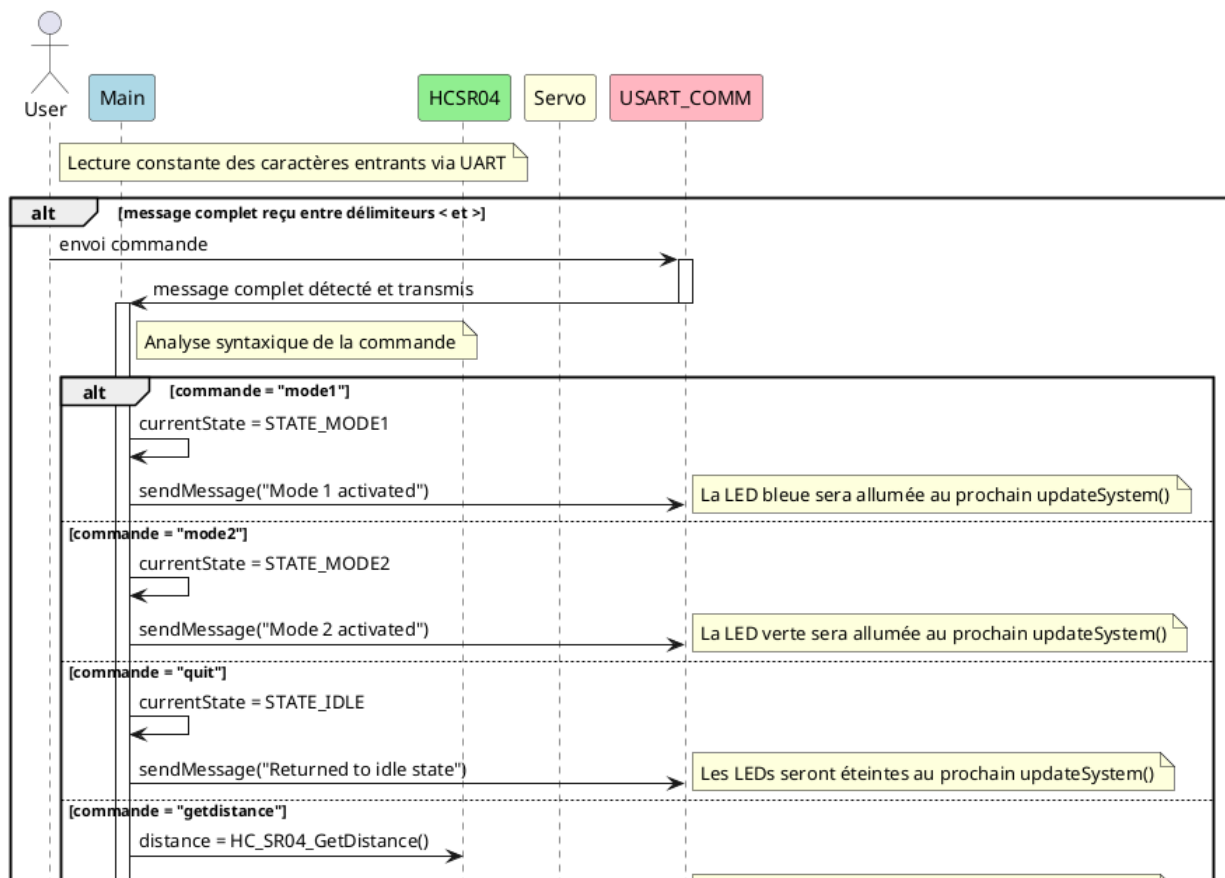
Ce diagramme montre :

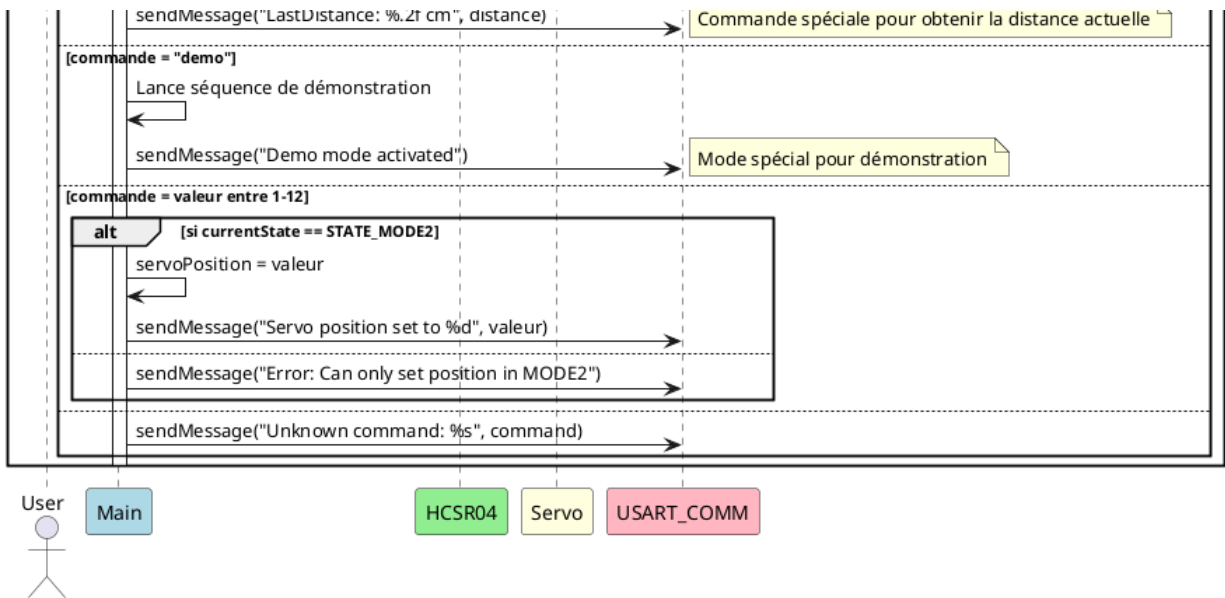
- La séquence d'initialisation des différents modules
- La structure de la boucle principale du système
- Les références vers les diagrammes plus détaillés
- L'interaction entre les acteurs principaux du système

Traitement des commandes utilisateur

Traitement des Commandes Utilisateur

Diagramme de séquence : Traitement des commandes utilisateur





Processus de réception et traitement des commandes utilisateur envoyées via l'interface série.

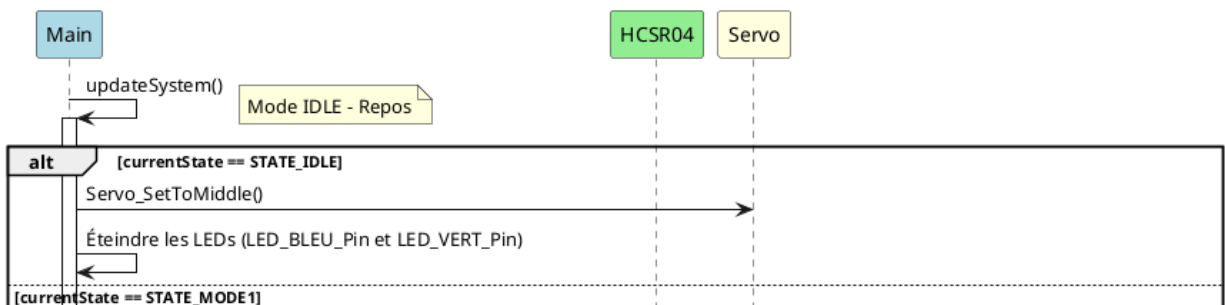
Il montre :

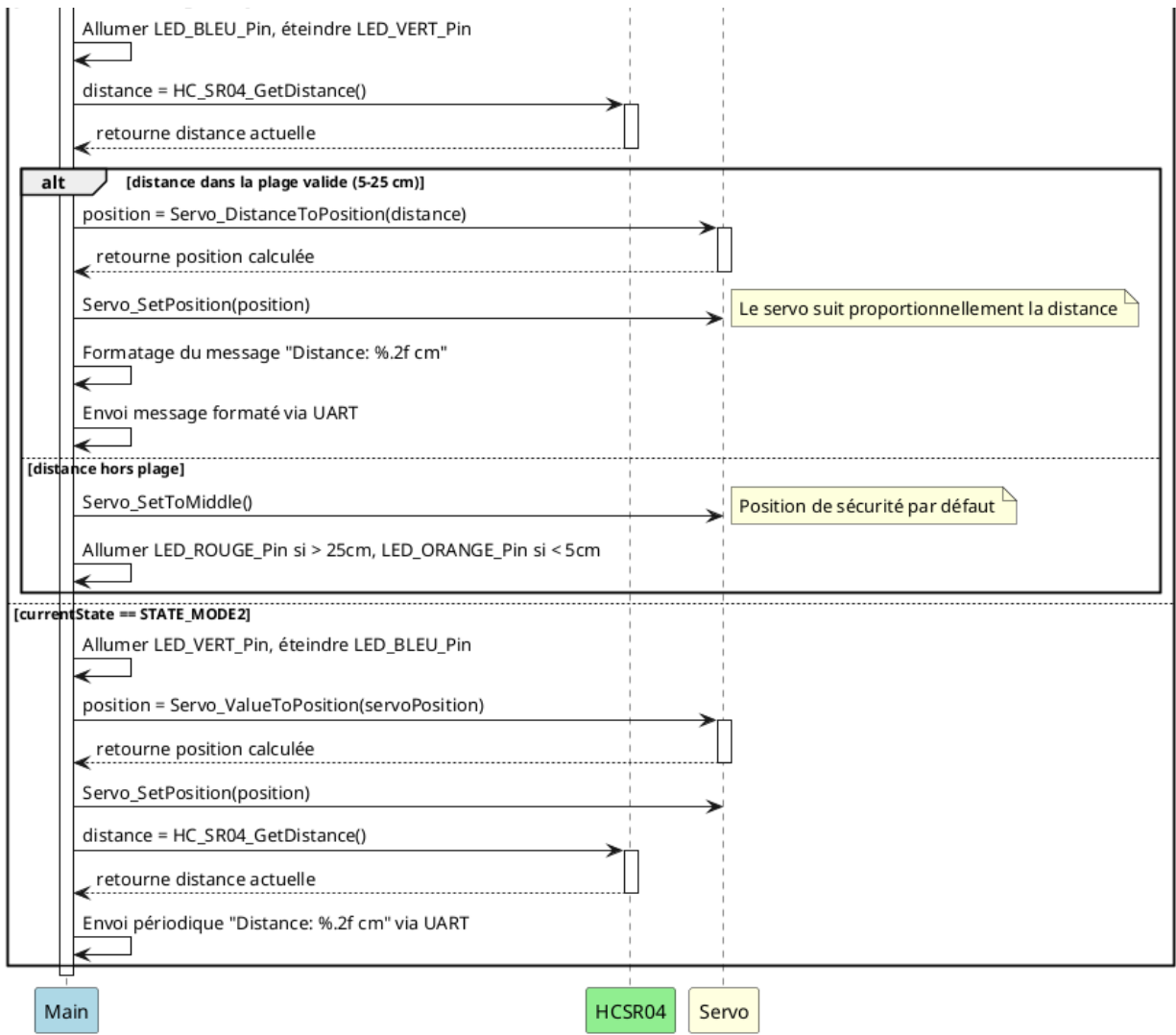
- La réception des commandes via l'interface UART
- L'analyse et l'interprétation des commandes reçues
- Les différentes actions selon le type de commande reçue
- Les messages de confirmation envoyés à l'utilisateur
- La gestion des erreurs pour les commandes invalides

Mise à jour du système selon l'état

Mise à Jour du Système Selon l'État

Diagramme de séquence : Mise à jour du système selon l'état





Comportement du système en fonction de son état actuel (IDLE, MODE1, MODE2).

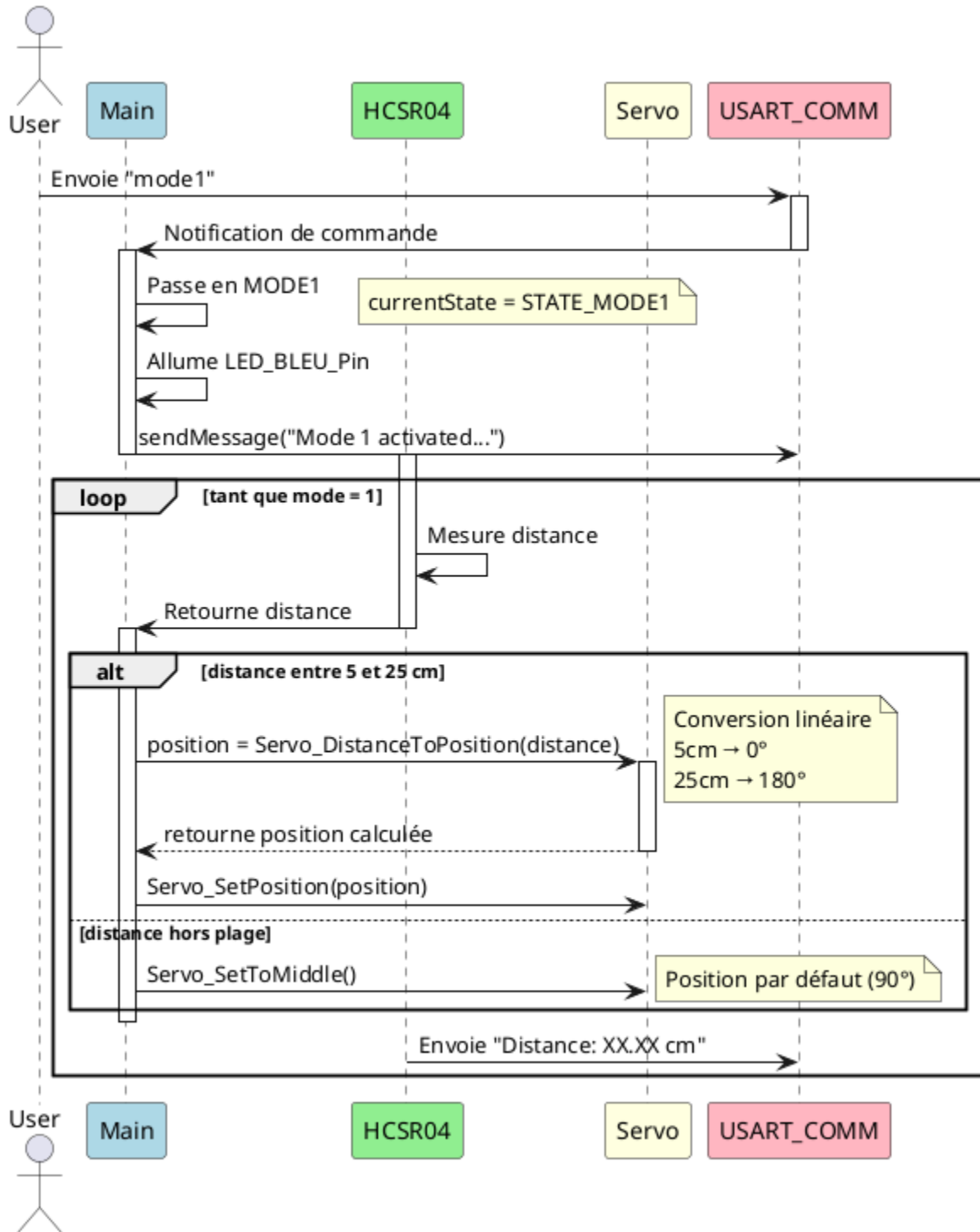
Il montre :

- Les actions spécifiques pour chaque état (IDLE, MODE1, MODE2)
- Le traitement des mesures de distance en MODE1
- La gestion des positions du servo-moteur selon le mode actif
- Le contrôle des LEDs pour indiquer visuellement l'état du système
- Les conditions de validation des distances mesurées

Mode 1 - Suivi de distance

Scénario : Mode 1 - Suivi de Distance

Scénario d'utilisation: Mode 1 - Suivi de distance



Projet Capteur à Actionneur - Avril 2025

Fonctionnement détaillé du Mode 1 où le servo-moteur suit la distance mesurée par le capteur.

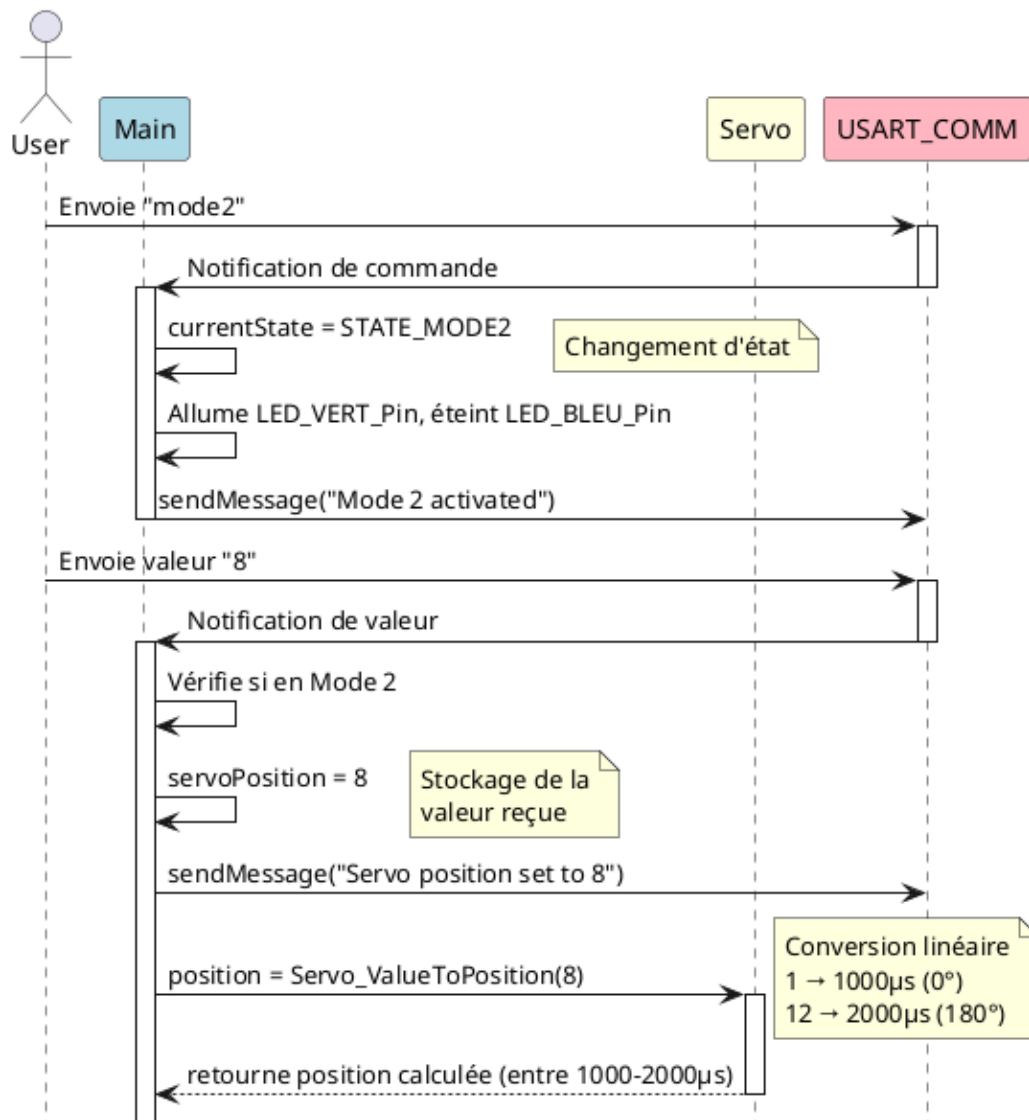
Ce scénario montre :

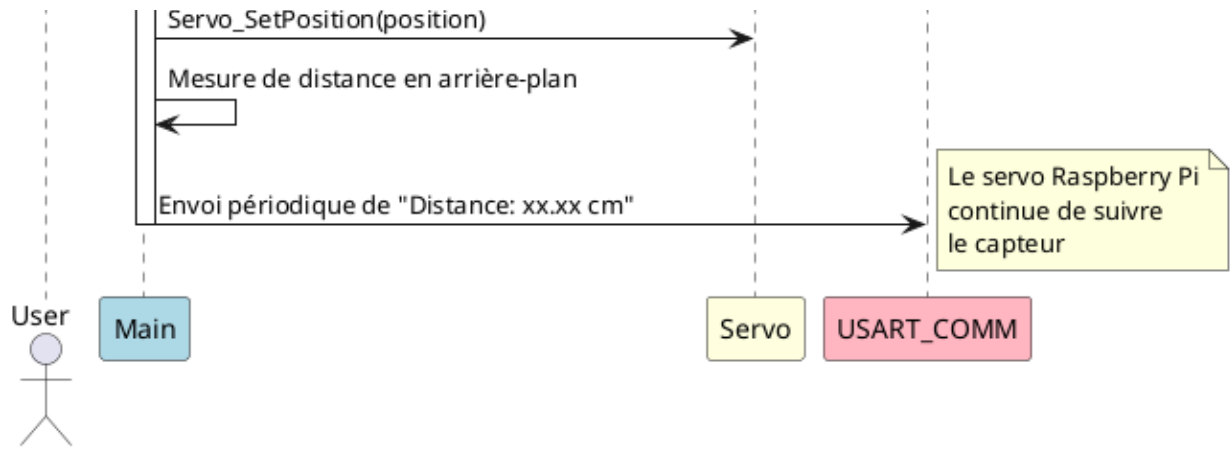
- L'activation du Mode 1 par l'utilisateur via UART
- La mesure continue de la distance par le capteur HC-SR04
- L'ajustement du servo en fonction de la distance mesurée
- L'envoi régulier des informations de distance par UART
- La gestion des valeurs hors plage (< 5 cm ou > 25 cm)

Mode 2 - Consigne utilisateur

Scénario : Mode 2 - Consigne Utilisateur

Scénario d'utilisation: Mode 2 - Consigne utilisateur





Projet Capteur à Actionneur - Mai 2025

Fonctionnement détaillé du Mode 2 où le servo-moteur suit les consignes de l'utilisateur.

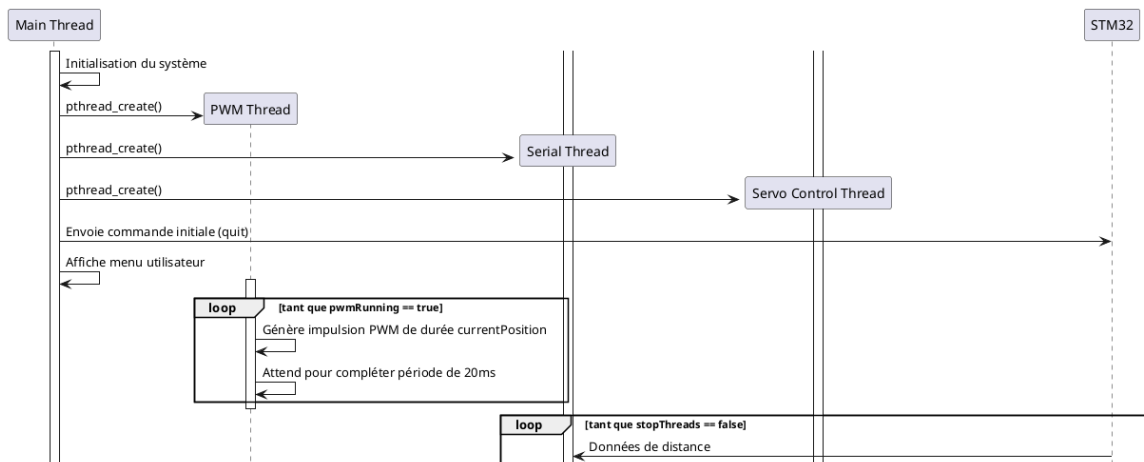
Ce scénario montre :

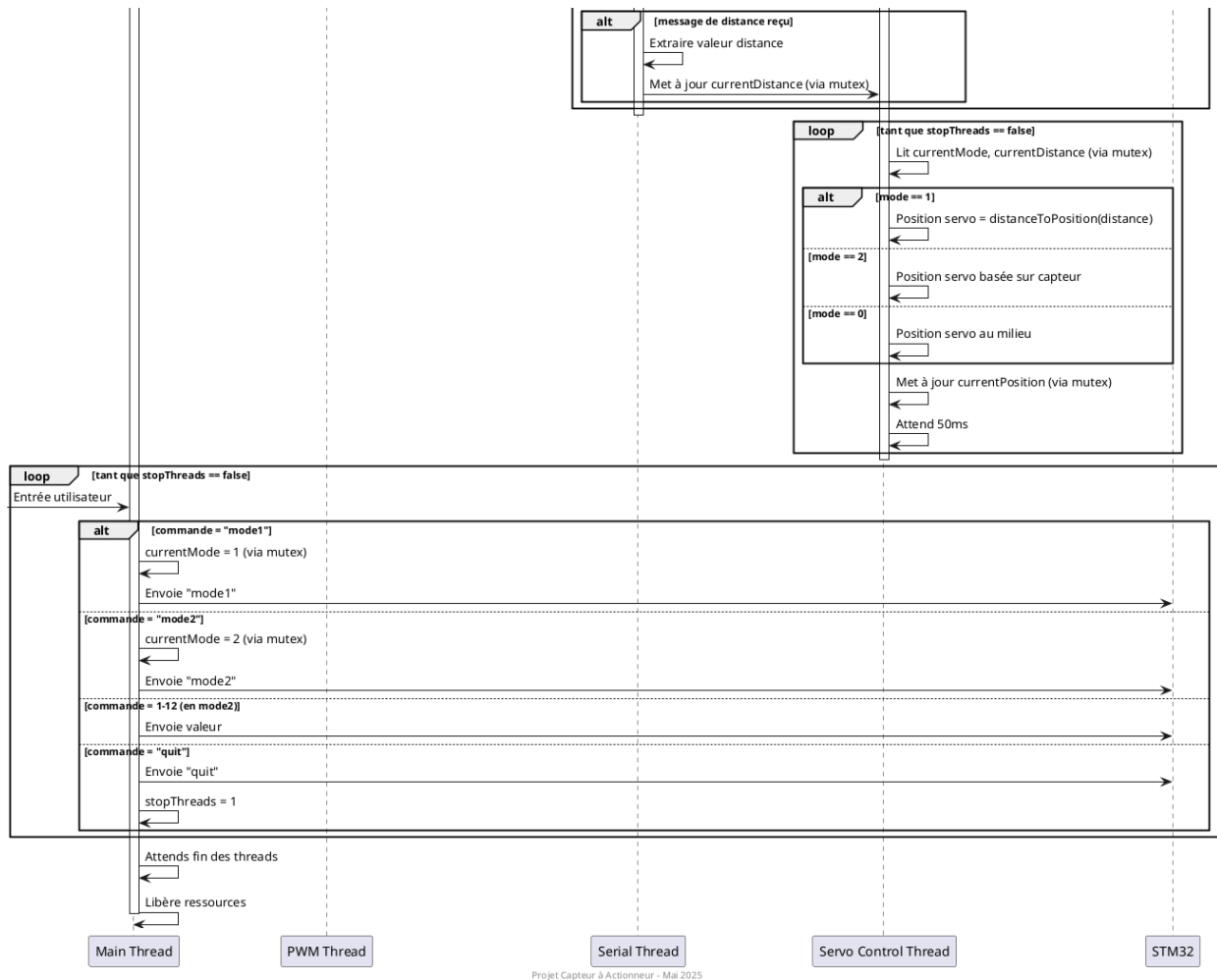
- L'activation du Mode 2 par l'utilisateur
- L'envoi d'une valeur de position par l'utilisateur
- Le processus de conversion de la valeur (1-12) en position servo
- Le positionnement du servo selon la consigne reçue
- Les confirmations renvoyées à l'utilisateur

Gestion des threads sur Raspberry Pi

Architecture Multi-thread sur Raspberry Pi

Diagramme de séquence : Threads sur Raspberry Pi





Organisation des différents threads sur la Raspberry Pi et leurs interactions.

Ce diagramme illustre :

- La création des différents threads au démarrage du programme
- Le thread PWM dédié à la génération du signal pour le servo-moteur
- Le thread de réception des données série provenant de la STM32
- Le thread de contrôle du servo-moteur selon le mode et les données reçues
- Le thread principal gérant l'interface utilisateur et l'envoi des commandes
- L'utilisation des mutex pour la synchronisation des accès aux variables partagées
- Les flux de communication entre la Raspberry Pi et la STM32

2.4 Brochage des différents éléments

Brochage du capteur HC-SR04

Broche	Connexion	Description
VCC	Alimentation 5V	Alimentation du capteur
GND	Masse	Référence de masse
Trig	PA8 (GPIO)	Signal de déclenchement de la mesure (sortie STM32)
Echo	PA9 (GPIO)	Signal de retour de l'écho (entrée STM32)

Brochage du servo-moteur (STM32)

Broche	Connexion	Description
VCC	Alimentation 5V	Alimentation du servo
GND	Masse	Référence de masse
Signal	PC6 (TIM3_CH1)	Signal PWM pour le contrôle de position

Brochage du servo-moteur (Raspberry Pi)

Broche	Connexion	Description
VCC	5V (Pin 4)	Alimentation du servo
GND	GND (Pin 6)	Référence de masse
Signal	GPIO 18 (Pin 12)	Signal PWM pour le contrôle de position

Brochage des LEDs

LED	Broche	Mode associé	Signification
LED Bleue	PD15	MODE1	Mode capteur actif
LED Verte	PD12	MODE2	Mode commande série actif

LED	Broche	Mode associé	Signification
LED Orange	PD13	Indication	Mesure valide (5-25 cm)
LED Rouge	PD14	Indication d'erreur	Mesure hors plage

Brochage de la Communication série STM32-PC

Broche	Connexion	Description
TX	PA2	Transmission de données
RX	PA3	Réception de données

Brochage de la Communication série STM32-Raspberry Pi (USART2)

Appareil	Fonction	Broche	Connexion
STM32	TX	PA2	Connecté à RX Raspberry Pi
	RX	PA3	Connecté à TX Raspberry Pi
Raspberry Pi	TX	GPIO 14 (Pin 8)	Connecté à RX STM32
	RX	GPIO 15 (Pin 10)	Connecté à TX STM32
<i>Note: Les masses (GND) des deux systèmes sont connectées ensemble pour garantir une référence commune</i>			

2.4 Configuration des différents éléments

Configuration du servo-moteur

Paramètre	Valeur	Description
Fréquence PWM	50 Hz	Fréquence du signal de contrôle (période 20ms)

Paramètre	Valeur	Description
Position minimale	1000 μ s	Largeur d'impulsion pour 0°
Position centrale	1500 μ s	Largeur d'impulsion pour 90°
Position maximale	2000 μ s	Largeur d'impulsion pour 180°
Résolution	1 μ s	Résolution du positionnement

Configuration du capteur HC-SR04

Paramètre	Valeur	Description
Durée trigger	10 μ s	Durée de l'impulsion de déclenchement
Plage de mesure	5-25 cm	Plage de distance valide pour l'application
Timeout	10 ms	Délai maximum d'attente pour l'écho
Timer utilisé	TIM1	Timer haute résolution pour la mesure temporelle
Pull-down	Activé	Sur la broche Echo pour stabiliser le signal

Configuration de la communication série

Paramètre	Valeur	Description
Interface STM32	USART2	Interface série utilisée sur le STM32
Interface Raspberry Pi	/dev/serial0	Port série de la Raspberry Pi
Vitesse	115200 bauds	Débit de transmission
Format de trame	8N1	8 bits de données, pas de parité, 1 bit de stop

Paramètre	Valeur	Description
Délimiteurs	'<', '>'	Caractères d'encapsulation des messages
Timeout UART	1000 ms	Timeout pour les opérations de transmission
Taille buffer	256 octets	Taille des buffers de réception et transmission

3. Modules développés

3.1. Module HC-SR04

Le module HC-SR04 gère l'interfaçage avec le capteur ultrasonique pour mesurer la distance.

Fonctionnalités principales

- Initialisation des broches GPIO et du timer
- Génération d'impulsions précises de 10µs pour le trigger
- Mesure du temps d'écho avec une résolution microseconde
- Conversion du temps d'écho en distance (cm)
- Filtrage des mesures hors plage (5-25 cm)
- Mise à jour périodique des mesures en Mode 1

3.2. Module Servo-moteur

Le module servo-moteur permet de contrôler la position angulaire d'un servo standard via un signal PWM.

Fonctionnalités principales

- Initialisation du timer PWM (TIM3)
- Positionnement précis du servo à une valeur spécifiée
- Conversion de distances (5-25 cm) en positions servo (1000-2000 µs)
- Conversion de valeurs utilisateur (1-12) en positions servo
- Position centrale par défaut (1500 µs, 90°)

Détails techniques

- Signal PWM généré à 50Hz (période de 20ms)
- Largeur d'impulsion variable entre 1ms (0°) et 2ms (180°)
- Position centrale à 1.5ms (90°)
- Résolution temporelle de 1µs

3.3. Module UART

Le module UART gère la communication série avec l'utilisateur, offrant une interface formatée pour les commandes et les messages d'état.

Fonctionnalités principales

- Envoi de messages formatés avec délimiteurs
- Réception non-bloquante de commandes
- Écho des caractères reçus
- Gestion des dépassements de buffer
- Support pour les commandes textuelles et numériques

Protocole de communication

Type de message	Format	Exemple	Description
Commande mode	<commande>	<mode1>	Change le mode de fonctionnement
Valeur servo	<valeur>	<8>	Positionne le servo en Mode 2
Information	<message>	<Distance: 15.32 cm>	Retour d'information
Erreur	<message d'erreur>	<Error: Message too long>	Indication d'une erreur

Commandes reconnues :

- `model` : Active le Mode 1 (suivi par capteur)
- `mode2` : Active le Mode 2 (commande manuelle)
- `quit` : Retourne au mode IDLE
- `1` à `12` : Valeurs pour positionner le servo en Mode 2
- `getdistance` : Demande la dernière distance mesurée
- `demo` : Lance une séquence de démonstration

3.4. Fonctions de démonstration

Pour valider le bon fonctionnement des différents modules, nous avons implémenté des fonctions de démonstration. Elles permettent de tester individuellement chaque composant (capteur HC-SR04, servo-moteur, UART) sans dépendre des autres parties du système.

Voici un résumé des fonctions de démonstration et des indicateurs de bon fonctionnement associés :

Module	Fonction de démonstration	Indicateur de bon fonctionnement
Capteur HC-SR04	<code>HC_SR04_Demo</code>	Affichage de LED indicatives : • LED orange : distance valide (5-25 cm) • LED rouge : distance hors plage ou erreur
Servo-moteur	<code>Servo_Demo</code>	Déplacement fluide entre les positions : • minimale (0°), • centrale (90°), • maximale (180°).
UART	<code>USART_Demo</code>	Test interactif : • Envoi d'un message de test : "Testing USART communication... Please respond with <response> to continue." • Réception et validation de la réponse utilisateur : "Response received: continuing demo."

3.5. Application principale

L'application principale orchestre tous les modules et implémente la machine à états qui gère le comportement du système.

Fonctionnalités principales

- Initialisation de tous les périphériques
- Boucle principale non-bloquante
- Traitement des commandes utilisateur
- Gestion des transitions entre états
- Mise à jour périodique de l'état du système

Stratégie de conception

- **Architecture modulaire** : Chaque module a une responsabilité claire et bien définie
- **Machine à états** : Gestion explicite des états et transitions pour simplifier la logique
- **Gestion non-bloquante** : Aucune fonction ne bloque l'exécution, permettant au système de réagir en temps réel
- **Extensibilité** : Facilité d'ajout de nouveaux modes ou comportements

3.6. Modules Raspberry Pi

La Raspberry Pi est organisée en plusieurs modules qui travaillent ensemble pour offrir les fonctionnalités requises. Chaque module gère un aspect spécifique du système.

3.6.1. Module GPIO

Ce module (fichiers gpio.c et gpio.h) gère l'accès aux broches GPIO de la Raspberry Pi en utilisant l'interface sysfs.

Fonctionnalités principales :

- Initialisation des broches GPIO et définition de leur direction (entrée/sortie)
- Contrôle de l'état des broches (écriture de valeurs 0 ou 1)
- Gestion du servo-moteur connecté au GPIO 18
- Interface par fichiers système pour un contrôle précis des broches

Fonctions importantes :

- `setupGPIO()` : Configure la broche du servo-moteur en sortie
- `exportGPIO()` : Rend la broche accessible via l'interface sysfs
- `setGPIONDirection()` : Définit si la broche est une entrée ou une sortie
- `writeGPIO()` : Écrit une valeur sur la broche (0 ou 1)

3.6.2. Module Serial

Ce module (fichiers `serial.c` et `serial.h`) gère la communication UART entre la Raspberry Pi et la STM32F4.

Fonctionnalités principales :

- Établissement de la connexion série à 115200 bauds, 8N1
- Envoi de commandes formatées à la STM32 avec les délimiteurs < et >
- Réception et interprétation des données de distance
- Analyse des messages reçus pour extraire les informations pertinentes

Fonctions importantes :

- `openSerialConnection()` : Ouvre et configure le port série
- `sendCommand()` : Envoie une commande formatée à la STM32
- `receiveDataThread()` : Thread dédié à la réception des données

Le module `serial` inclut un thread dédié qui s'exécute en arrière-plan et analyse en continu les données reçues de la STM32. Il extrait notamment les informations de distance mesurées par le capteur à ultrasons et les met à disposition des autres modules via des variables partagées protégées par mutex.

3.6.3. Module Servo

Ce module (fichiers `servo.c` et `servo.h`) gère le contrôle du servo-moteur connecté à la Raspberry Pi.

Fonctionnalités principales :

- Génération de signaux PWM précis via GPIO logiciel
- Conversion de distances mesurées en positions servo
- Conversion de valeurs utilisateur en positions servo
- Thread dédié à la génération du signal PWM

Fonctions importantes :

- `distanceToPosition()` : Convertit une distance mesurée en position du servo
- `valueToPosition()` : Convertit une valeur utilisateur en position du servo
- `pwmThread()` : Thread qui génère le signal PWM
- `servoControlThread()` : Thread qui contrôle le servo selon le mode actuel

Le module servo utilise deux threads distincts :

- Un thread PWM qui génère précisément les impulsions pour le servo-moteur
- Un thread de contrôle qui détermine la position du servo selon le mode et les données disponibles

3.6.4. Module Main

Ce module (fichiers `main.c` et `common.h`) orchestre l'ensemble du système sur la Raspberry Pi.

Fonctionnalités principales :

- Initialisation de tous les modules
- Création et gestion des threads
- Interface utilisateur en ligne de commande
- Traitement des commandes utilisateur
- Gestion des modes de fonctionnement
- Nettoyage des ressources à la fermeture

Fonctions importantes :

- `main()` : Point d'entrée du programme
- `handleUserInput()` : Gère l'interface utilisateur

- `printHelp()` : Affiche l'aide sur les commandes disponibles
- `cleanup()` : Libère les ressources et arrête proprement le système

Le fichier `common.h` définit les constantes partagées et les variables globales utilisées par tous les modules, notamment :

- Numéros de broches GPIO
- Limites de positions pour le servo-moteur
- Limites de distances pour le capteur
- Variables partagées comme le mode actuel, la distance mesurée, etc.

4. Difficultés rencontrées et solutions

Problème	Description	Solution mise en œuvre
Précision des mesures	Fluctuations importantes dans les mesures du capteur HC-SR04, rendant le positionnement du servo instable	• Ajout d'un pull-down sur la broche Echo • Utilisation de TIM1 pour mesurer avec une résolution de 1µs • Implémentation de timeouts (10ms) • Filtrage des valeurs hors plage (5-25 cm)
Communication série bloquante	Les premières implémentations de la communication	• Réception non-bloquante des caractères • Gestion d'état

Problème	Description	Solution mise en œuvre
	série bloquaient l'exécution principale	pour reconstituer les commandes • Utilisation de délimiteurs '<' et '>' • Buffer statique pour conserver l'état entre appels
Stabilité du servo	Servo instable lors de petits changements de distance	• Limitation de la fréquence de mise à jour (100ms) • Position centrale par défaut en cas d'erreur
Dysfonctionnement du servo-moteur	Après plusieurs essais, le servo-moteur a cessé de répondre correctement aux commandes	• Test valide avec un autre servo-moteur • Vérification de la connexion électrique • Changement d'un fil de connexion

Problème	Description	Solution mise en œuvre
Communication STM32-Raspberry Pi	Difficultés lors de l'échange de données entre les deux plateformes	• Utilisation d'un adaptateur de niveau logique • Mise en place d'un protocole robuste avec délimiteurs • Gestion d'erreurs et retransmission en cas d'échec • Ajout d'un petit délai (1ms) entre l'envoi de chaque caractère pour éviter la perte de données
Génération PWM Raspberry Pi	Difficultés à générer un signal PWM précis pour le servo sur Raspberry Pi	• Implémentation d'un thread dédié au PWM • Utilisation de timers logiciels avec usleep() • Compensation des délais d'exécution • Calcul précis des temps

Problème	Description	Solution mise en œuvre
		d'attente pour maintenir une période constante de 20ms
Synchronisation multi-thread	Problèmes d'accès concurrent aux variables partagées entre les threads	• Utilisation de mutex pour protéger les accès aux données partagées • Isolation des fonctions critiques • Optimisation des sections critiques pour minimiser les temps de verrouillage
Création de la machine à états	Implémentation de la logique de transition entre les différents modes	• Conception claire des états et transitions • Documentation détaillée avec diagrammes • Tests systématiques des transitions

Problème	Description	Solution mise en œuvre		
Configuration UART Raspberry Pi	Problèmes de configuration du port série sur la Raspberry Pi	• Suppression du paramétrage VMIN et VTIME qui causait des blocages • Évitement du mode non-bloquant qui engendrait des pertes de données • Utilisation de tcflush() pour vider les buffers avant utilisation • Mauvaise configuration du port série • Ouverture d'une communication parallèle bloquant l'uart dans l'application	Interface Raspberry	• Problèmes d'affichage concurrent lors de l'écriture de commande et lecture de données • Répétition de l'envoi du mode actif, rendant moins lisible l'interface

5. Interaction Raspberry Pi - STM32

L'interaction entre la Raspberry Pi et le STM32 constitue un aspect essentiel de ce projet, permettant de créer un système distribué où chaque plateforme joue un rôle complémentaire.

5.1 Architecture du système global

Le système global suit une architecture maître-esclave où la Raspberry Pi agit comme maître et le STM32 comme esclave :

Composants principaux :

- **STM32F407** : Gère le capteur HC-SR04 et son propre servo-moteur
- **Raspberry Pi** : Contrôle un second servo-moteur et sert d'interface utilisateur, peut contrôler le servo-moteur de la STM32F407
- **Communication UART** : Lien bidirectionnel entre les deux plateformes
- **Interface utilisateur** : Console en mode texte sur la Raspberry Pi

Cette architecture présente plusieurs avantages :

- **Répartition des tâches** : Le STM32 s'occupe de l'interfaçage avec le capteur à ultrasons, tandis que la Raspberry Pi gère l'interface utilisateur et le traitement de plus haut niveau
- **Redondance** : Si l'une des plateformes rencontre un problème, l'autre peut continuer à fonctionner partiellement
- **Flexibilité** : L'architecture permet d'ajouter facilement de nouvelles fonctionnalités sur l'une ou l'autre plateforme
- **Apprentissage** : Cette configuration permet d'explorer les contraintes et avantages des deux types de systèmes
- **STM32F407 VGT** avec capteur HC-SR04 et servo-moteur
- **Raspberry Pi** avec son propre servo-moteur
- **Communication UART** entre les deux systèmes
- **Interface utilisateur** sur la Raspberry Pi

5.2 Fonctionnalités de la Raspberry Pi

Dans ce système distribué, la Raspberry Pi remplit plusieurs rôles :

- 1 **Interface utilisateur** : La Raspberry Pi fournit une interface en ligne de commande permettant à l'utilisateur de contrôler l'ensemble du système. Cette interface propose des commandes intuitives pour changer de mode, définir des positions ou récupérer des informations.

- 2 **Point de contrôle central** : La Raspberry Pi sert de coordinateur pour l'ensemble du système, transmettant les commandes au STM32 et gérant les états globaux du système.
- 3 **Contrôle du servo-moteur** : La Raspberry Pi contrôle son propre servo-moteur qui réagit soit aux distances mesurées par le capteur (transmises par le STM32), soit aux commandes utilisateur selon le mode actif.
- 4 **Traitement des données** : La Raspberry Pi récupère les données de distance depuis le STM32, les traite et les affiche dans un format compréhensible pour l'utilisateur.
- 5 **Diagnostic et débogage** : La Raspberry Pi fournit des outils pour diagnostiquer le fonctionnement du système, comme la commande `getdistance` pour vérifier la dernière mesure ou le programme `minicom.c` pour tester la communication série.

5.3 Modules logiciels Raspberry Pi

L'application sur Raspberry Pi est développée en C et organisée en plusieurs modules fonctionnels :

Module	Fichiers	Responsabilités
Principal	main.c, common.h	• Initialisation du système • Création des threads • Interface utilisateur • Traitement des commandes • Définition des constantes et variables partagées
Communication série	serial.c, serial.h	• Établissement de la connexion UART avec le STM32 • Formatage et envoi des commandes • Réception et traitement des données • Extraction des informations de

Module	Fichiers	Responsabilités
		distance
Servo-moteur	servo.c, servo.h	• Génération du signal PWM pour le servo-moteur • Conversion des distances en positions • Conversion des valeurs utilisateur en positions • Synchronisation du servo selon le mode actif
GPIO	gpio.c, gpio.h	• Configuration des broches GPIO • Contrôle des broches de sortie • Gestion du servo-moteur via GPIO 18
Utilitaires	test_uart.c, minicom.c	• Outils de test pour la communication UART • Diagnostics du système • Vérification des connexions matérielles

5.4 Modes de fonctionnement

Le système sur Raspberry Pi supporte les mêmes modes que le STM32, mais avec quelques particularités :

Mode	Comportement STM32	Comportement Raspberry Pi
Mode IDLE	• LEDs éteintes • Servo en position centrale • Mesure de distance inactive	• Servo en position centrale • Interface utilisateur active • Attente de commandes

Mode	Comportement STM32	Comportement Raspberry Pi
Mode 1	• LED bleue allumée • Servo suit la distance mesurée • Envoi régulier des mesures	• Servo suit également la distance mesurée • Affichage des mesures reçues • Synchronisation avec le servo du STM32
Mode 2	• LED verte allumée • Servo suit les commandes utilisateur (1-12) • Continue de mesurer et envoyer les distances	• Servo continue de suivre la distance mesurée • Transmet les commandes de position au STM32 • Affiche les distances et confirmations

Cette configuration crée un système où, en Mode 2, les deux servo-moteurs adoptent des comportements différents : le servo du STM32 suit les commandes manuelles, tandis que le servo de la Raspberry Pi continue de suivre les mesures de distance.

5.5 Protocole de communication

La communication entre la Raspberry Pi et le STM32 utilise un protocole série simple mais robuste :

Caractéristiques du protocole :

- Messages encapsulés entre délimiteurs < et >
- Format texte pour faciliter le débogage
- Commandes courtes et explicites
- Confirmations systématiques des commandes
- Transmission régulière des mesures de distance

Types de messages échangés :

Direction	Type de message	Exemple	Description
Raspberry Pi → STM32	Commande de mode	<mode1>	Change le mode de fonctionnement
Raspberry Pi → STM32	Position servo	<8>	Définit la position du servo STM32 (1-12)
Raspberry Pi → STM32	Demande de distance	<getdistance>	Demande la dernière mesure de distance
STM32 → Raspberry Pi	Mesure de distance	<Distance: 15.32 cm>	Transmet une mesure de distance
STM32 → Raspberry Pi	Confirmation	<Mode 1 activated>	Confirme une commande reçue
STM32 → Raspberry Pi	Erreur	<Error: Invalid command>	Signale une erreur dans une commande

Pour assurer une communication fiable, la fonction `sendCommand()` a été implémentée avec un mécanisme spécial : elle envoie chaque caractère individuellement avec un petit délai entre chaque envoi. Cette approche, bien que plus lente, évite les pertes de données qui peuvent survenir lorsque le buffer du récepteur est saturé.

```
void sendCommand(int fd, const char* command) {
    if (fd < 0) return;

    // Formater la commande
    char formattedCommand[64];
    snprintf(formattedCommand, sizeof(formattedCommand), "<%s>", c

    // Assurez-vous que chaque caractère est envoyé correctement
    for (int i = 0; i < strlen(formattedCommand); i++) {
        write(fd, &formattedCommand[i], 1);
        // Petit délai entre chaque caractère
```

```

        usleep(1000); // 1ms
    }

    printf("Commande envoyée: %s\n", command);
}

```

5.6 Implémentation multi-thread

Une des particularités de l'implémentation sur Raspberry Pi est l'utilisation de plusieurs threads pour gérer différentes tâches en parallèle :

- 1 **Thread PWM** : Ce thread génère le signal PWM pour contrôler le servo-moteur. Il calcule précisément les timings nécessaires pour produire une impulsion de la durée adéquate toutes les 20ms (50Hz).
- 2 **Thread de réception série** : Ce thread surveille en permanence le port série pour recevoir et traiter les messages provenant du STM32, notamment les mesures de distance.
- 3 **Thread de contrôle servo** : Ce thread calcule la position que doit prendre le servo en fonction du mode actif et des données disponibles (distance mesurée ou commande utilisateur).
- 4 **Thread principal** : Ce thread gère l'interface utilisateur, traite les commandes et les transmet au STM32.

Pour coordonner ces threads et éviter les problèmes d'accès concurrent, le programme utilise des mutex qui protègent les variables partagées comme le mode actuel, la distance mesurée ou la position du servo.

```

// Exemple d'utilisation de mutex pour protéger l'accès aux varia
pthread_mutex_lock(&mutex);
int mode = currentMode;
float distance = currentDistance;
pthread_mutex_unlock(&mutex);

// Traitement avec les valeurs récupérées

```

```

if (mode == 1) {
    // Utilisation des données en mode 1
} else if (mode == 2) {
    // Utilisation des données en mode 2
}

```

Cette architecture multi-thread permet au système de gérer plusieurs tâches simultanément, comme la génération précise du signal PWM pendant que les données série sont reçues et que l'utilisateur entre des commandes.

5.7 Outils de diagnostic

Pour faciliter le développement et le débogage du système, plusieurs outils de diagnostic ont été créés :

Outil	Fichier	Objectif
Test UART	test_uart.c	• Vérifie la configuration et le fonctionnement de la communication série • Envoie un message de test et affiche la réponse • Aide à identifier les problèmes de connexion
Mini-terminal	minicom.c	• Outil simplifié similaire à minicom • Envoie une commande spécifique et affiche toutes les réponses • Montre chaque caractère reçu pour débogage à bas niveau • Identifie les messages complets entre délimiteurs < et >
Commande getdistance	Dans main.c	• Demande et affiche la dernière distance mesurée • Permet de vérifier le bon fonctionnement du

Outil	Fichier	Objectif
		capteur • Affiche les mesures dans un format coloré pour meilleure lisibilité

Ces outils ont été particulièrement utiles pour identifier et résoudre les problèmes de communication entre la Raspberry Pi et le STM32, notamment les questions de timing, de formatage des messages et de configuration du port série.

```
// Extrait du programme minicom.c pour le diagnostic
printf("Envoi de  puis affichage de toutes les réponses...\n");

// Envoyer model
const char *cmd = "\r\n";
write(fd, cmd, strlen(cmd));
tcdrain(fd);

// Lecture des réponses
printf("Réponses:\n");

// Mode non-bloquant
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);

// Écouter pendant 5 secondes
time_t end_time = time(NULL) + 5;

char buffer[256] = {0};
int bufferIndex = 0;
int inMessage = 0;

while (time(NULL) < end_time) {
    char c;
    if (read(fd, &c, 1) > 0) {
        // Afficher chaque caractère
```

```

printf("%c", (c >= 32 && c <= 126) ? c : '.');

// Construire des messages complets entre < et >
if (c == '<') {
    inMessage = 1;
    bufferIndex = 0;
    buffer[0] = c;
    bufferIndex++;
} else if (c == '>' && inMessage) {
    buffer[bufferIndex] = c;
    buffer[bufferIndex + 1] = '\0';
    printf("\nMessage complet: %s\n", buffer);
    inMessage = 0;
} else if (inMessage && bufferIndex < sizeof(buffer) - 2)
    buffer[bufferIndex] = c;
    bufferIndex++;
}

fflush(stdout);
}
usleep(5000);
}

```

6. Conclusion et perspectives

Ce projet a permis de développer un système complet intégrant :

1. **Architecture multi-systèmes** : Collaboration entre un microcontrôleur STM32 et une Raspberry Pi pour créer un système distribué de contrôle.
2. **Communication inter-système** : Utilisation de l'UART pour permettre l'échange d'informations entre les deux plateformes.
3. **Interface capteur-actionneur** : Traitement des données du capteur ultrason pour contrôler les servo-moteurs en temps réel.
4. **Machine à états robuste** : Gestion explicite des différents modes de fonctionnement avec transitions sécurisées.
5. **Interface utilisateur bi-plateforme** : Interaction possible via l'interface série du

STM32 ou via l'application console de la Raspberry Pi.

6. **Programmation multi-thread** : Utilisation de plusieurs threads sur Raspberry Pi pour gérer différentes tâches en parallèle.
7. **Développement modulaire** : Architecture logicielle organisée en modules réutilisables et bien séparés.

Améliorations futures

Plusieurs pistes d'amélioration pourraient être explorées :

- **Interface graphique** : Développement d'une interface graphique sur la Raspberry Pi pour visualiser les mesures de distance et contrôler les actionneurs de manière plus intuitive.
- **Traitement avancé des données** : Implémentation d'algorithmes de filtrage pour améliorer la précision des mesures et réduire les variations aléatoires.
- **Extension des capteurs** : Ajout de capteurs supplémentaires (température, humidité) pour enrichir les fonctionnalités et créer un système plus complet.
- **Communication sans fil** : Remplacement de la liaison UART par une communication Bluetooth ou WiFi pour plus de flexibilité dans le placement des composants.
- **Enregistrement de données** : Implémentation d'un système de logging et d'analyse des données sur la Raspberry Pi pour suivre l'évolution des mesures dans le temps.
- **Serveur/client** : Développement de l'application sous forme d'un serveur/client pour permettre un contrôle à distance via un navigateur web ou une application mobile.
- **Historisation des mesures** : Enregistrement des 200 dernières valeurs de distance et génération de fichiers pour analyse statistique ou visualisation graphique.
- **Amélioration du PWM sur Raspberry Pi** : Utilisation d'une bibliothèque dédiée